Programação Orientada por Objectos

Classes simples

Helder Daniel

v2025/6 1.1

Introdução às classes

- Estes slides apresentam uma introdução muito breve às classes em Java

- Estes conteúdos são introduzidos e desenvolvido nas aulas teóricas 1 e 2

Introdução às classes

- Uma classe tem:
  - uma representação de dados: **as variáveis de instância**
  - operações sobre esses dados

- isto é as classes têm comportamento

- Todas as operações sobre os dados da classe
- são implementadas em funções pertencentes à classe: **os métodos**

- Por exemplo uma classe **Point** em 2D terá **duas variáveis de instância** para guardar as coordenadas x e y

- E operações sobre pontos, por exemplo o **método**:

  double distancia(Vetor that)

Introdução às classes

- Uma classe tem:
  - uma representação de dados: **as variáveis de instância**
  - operações sobre esses dados

- isto é as classes têm comportamento

- Todas as operações sobre os dados da classe
- são implementadas em funções pertencentes à classe: **os métodos**

- Por exemplo uma classe **Point** em 2D terá **duas variáveis de instância** para guardar as coordenadas x e y

- E operações sobre pontos, por exemplo o **método**:

  double distancia(Point that)

Exemplo classe Point

```
class Point {
   private double x, y;









}



class Client {
   public static void main(String[] args) {


      p0.x = 1.2; //ERRO
   }
}
```

Variáveis de instância com visibilidade **private**

para evitar acesso direto do **Client** aos dados

o vetor apenas pode ser manipulado pelas operações definidas nos métodos

Erro

Exemplo classe Point

```
class Point {
   private double x, y;

   public Point(double x, double y) {   this.x = x;     this.y = y; }

}

class Client {
   public static void main(String[] args) {
      Point p0 = new Point(2.5, 4);

   }
}
```

A inicialização das variáveis da classe é efetuada pelo construtor

Tem de ser **público** para poder ser chamado no Client

O construtor é chamado na classe Client

Exemplo classe Point

```
class Point {
   private double x, y;

   public Point(double x, double y) {   this.x = x;     this.y = y; }

   public double distance(Point that) { return  //… }

}


class Client {
   public static void main(String[] args) {
      Point p0 = new Point(2.5, 4);
      Point p1 = new Point(-3, 2.1);
      System.out.println(p0.distance(p1));
   }
}
```

As operações sobre os dados da classe são **efetuadas por funções da classe**

Nunca na classe Client

Client chama o método **distance()** que é público

Exemplo classe Point

```
class Point {
   private double x, y;

   public Point(double x, double y) {   this.x = x;     this.y = y; }

   public double distance(Point that) {
            double dx = x - that.x;
            double dy = y - that.y;
            return Math.sqrt(dx*dx + dy*dy);
     }
```

Assim todo o código relacionado com a classe **Point** está no mesmo lugar

na classe Point

o código fica mais claro

facilita a manutenção

Exemplo classe Point

```
class Point {
   private double x, y;

   public Point(double x, dou

   public double distance(Po

   public String toString() { return "(" + this.x + "," + this.y + ")"; }


}
```

Se for necessário imprimir o valor das variáveis do **Point** implementa-se na classe **Point** o método:

String toString()

que retorna uma string
Não se imprime na classe

```
class Client {
   public static void main(String[] args) {
      Point p0 = new Point(2.5, 4);

      System.out.println(p0);
   }
}
```

Na classe **Client** ao imprimir-se o Point

é automaticamente chamado este método

Só se deve imprimir na classe que implementa o UI
neste caso Client

Exemplo classe Point

Excecionalmente, enquanto não estudarmos **Exceptions**

O construtor pode imprimir uma mensagem de erro e terminar o programa.
Exemplo classe para Pontos apenas do 1º quadrante (x e y >= 0)

```
public Ponto1Quad(double x, double y) {

        if (x < 0 || y < 0) {
                System.out.println("x e y tem de ser >= 0");

                System.exit(0);     //Tem de ser zero para q o Mooshak aceite
        }

}
```

```
}
```

Exemplo classe Point

```
class Point {
   private double x, y;

   public Point(double x, double y) {   this.x = x;     this.y = y; }

   public double distance(Point that) { return  //… }

   public String toString() { return "(" + this.x + "," + this.y + ")"; }
   public double x() { return this.x; }
   public double y() { return this.y; }
}



class Client {
   public static void main(String[] args) {
      Point p0 = new Point(2.5, 4);

      System.out.println(p0.x());
   }
}
```

Se for necessário ler o valor das variáveis de instância implementam-se métodos assessores de leitura

Nota: não é necessário em muitos casos, pq as operações são implementadas dentro da classe

Exemplo classe Point

```
class Point {
   private double x, y;

   public Point(double x, double y) {   this.x = x;     this.y = y; }

   public double distance(Point that) { return  //… }

   public String toString() { return "(" + this.x + "," + this.y + ")"; }
   public double x() { return this.x; }
   public double y() { return this.y; }
   public void x(double x) { this.x = x; }
   public void y(double y) { this.y = y; }
}

class Client {
   public static void main(String[] args) {
      Point p0 = new Point(2.5, 4);

      System.out.println(p0.x());
   }
}
```

Evitar **accessores de escrita**, para alterar os valores das variáveis.

O variáveis de instância das classes **devem apenas ser alteradas pelos métodos que implementam as operações**

muito raramente diretamente

Exemplo classe Point

```
class Point {
   private double x, y;

   public Point(double x, double y) {   this.x = x;     this.y = y; }

   public double distance(Point that) { return  //… }




}



class Client {
   public static void main(String[] args) {
      Point p0 = new Point(2.5, 4);
      Point p1 = new Point(1.5, -2);

   }
}
```

Cada objeto da classe **Point** tem (ou pode ter) valores diferentes em x e y

Exemplo classe Point

```
class Point {
   private double x, y;

   public Point(double x, double y) {   this.x = x;     this.y = y; }

   public double distance(Point that) { return  //… }

    public static double metodoEstatico() {
            this.x; //ERRO
            //...
            }
}


class Client {
   public static void main(String[] args) {
      Point p0 = new Point(2.5, 4);
      Point p1 = new Point(1.5, -2);
      Point.metodoEstatico();
      p0.metodoEstatico();      //ERRO
   }
}
```

Métodos **static** não conseguem aceder ás variáveis de instância dos objetos

São chamados para a classe e não para o objeto

Por isso não sabem qual o valor de x e y em p0 ou p1

Exemplo classe Point

```
class Point {
   private double x, y;

   public Point(double x, double y) {   this.x = x;     this.y = y; }

   public double distance(Point that) { return  //… }

    public static double metodoEstatico() {
            this.x; //ERRO
            //...
            }
}


class Client {
   public static void main(String[] args) {
      Point p0 = new Point(2.5, 4);
      Point p1 = new Point(1.5, -2);
      Point.metodoEstatico();
      p0.metodoEstatico();      //ERRO
   }
}
```

Mais tarde veremos qual a utilidade dos métodos static

Para já o único método static é **main()**